



UMT

University of Management
and Technology

IDS-WSN

Intrusion Detection System
in wsn



Intrusion Detection System in Wireless Sensor Networks (IDS-WSN)

KALEEMULLAH YUNAS, F2022332074 - Department of AI, University of Management and Technology

ASADULLAH MASOOD, F2022332082 - Department of AI, University of Management and Technology

MUHAMMAD DAWOOD JAFFAR, F2022332064 - Department of AI, University of Management and Technology

Abstract

Wireless Sensor Networks (WSNs) sit at an uncomfortable intersection: they are central to modern infrastructure - smart cities, industrial control, environmental monitoring, healthcare - yet notoriously hard to secure. Nodes run on batteries, process data on slow microcontrollers, and often sit in physically accessible locations. Network traffic is heavily skewed toward normal operation, which makes training a reliable intrusion detector genuinely difficult. This study builds and evaluates a machine learning-based Intrusion Detection System for WSNs using the WSN-DS Kaggle dataset: 365,788 labeled network records spanning five traffic classes (Blackhole, Grayhole, Flooding, TDMA, and Normal). A complete ML pipeline was developed covering data preprocessing, exploratory analysis, correlation-based feature selection, three class balancing strategies (SMOTE, SMOTE-Tomek, SMOTE-ENN), and training of seven classifier variants including Logistic Regression tuned via GridSearchCV and three Naive Bayes implementations. The tuned Logistic Regression achieved 95.4% accuracy with a 99.0% ROC-AUC. GaussianNB performed as the best lightweight option at 97.1% AUC and 94.4% precision. Both models outperform rule-based detection baselines, particularly for minority attack classes where the class imbalance is most severe.

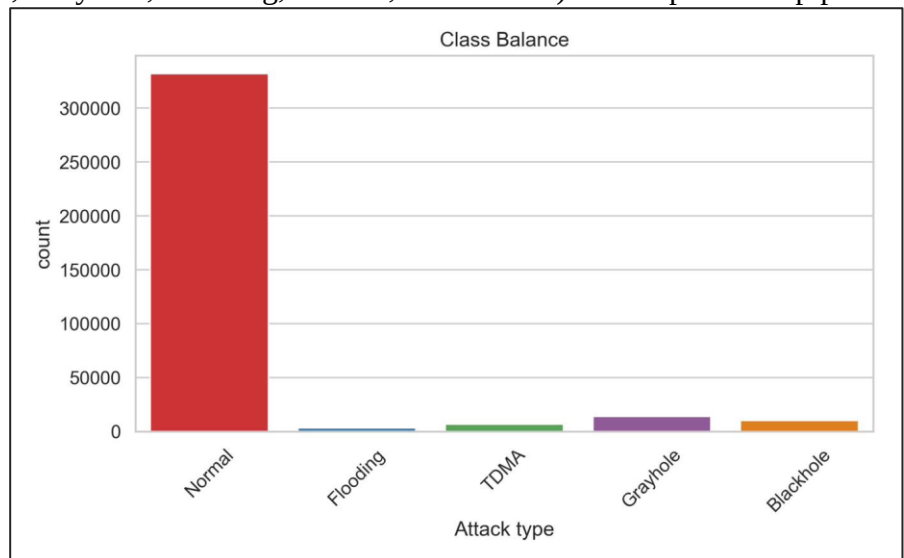


Figure 1 Class distribution in WSN-DS. Normal traffic dominates at ~95% of all records

1. INTRODUCTION

Wireless Sensor Networks have moved well past the research lab. Today they monitor industrial pipelines, track patients in hospitals, manage traffic signals, and support military operations in environments where wired infrastructure is simply not practical. The basic setup is familiar: small sensor nodes collect data from the physical environment, process it locally, and route it wirelessly through multi-hop paths toward a base station or gateway.

The problem is that these nodes are constrained in nearly every dimension that matters for security. They run on coin cells or harvest tiny amounts of ambient energy, so expensive cryptographic operations are off the table. Their processors clock in the megahertz range with kilobytes of RAM. Many nodes are deployed in remote or physically exposed locations, making hardware tampering a realistic threat. Network topologies shift as nodes fail or exhaust their batteries, which makes traditional perimeter defense largely useless.

Four WSN-specific attack types are particularly well-documented in the literature and are represented in the WSN-DS dataset used in this project:

Blackhole: A compromised node advertises the shortest path to the base station but silently drops all forwarded packets. Simple yet effective - it can partition network segments without any visible traffic anomaly at the attacker.

Grayhole: More sophisticated than Blackhole, the attacker drops packets selectively based on destination address or timing. The attack pattern is designed to look like legitimate lossy communication, making it the hardest of the four types to detect.

Flooding: The attacker generates massive volumes of spurious RREQ or data traffic, exhausting target nodes' buffers and consuming shared bandwidth. This is essentially a distributed denial-of-service attack adapted for constrained WSN hardware.

TDMA violations: Time-division multiple access protocols allocate specific time slots to each node for transmission. An attacker transmitting outside its assigned slot causes collisions and degrades throughput for all neighboring nodes.

Signature-based intrusion detection works well against documented attacks but fails against new variants and generates high false-positive rates in dynamic WSN environments. Machine learning approaches learn the statistical structure of normal and attack traffic, making them more adaptable. The core question this project addresses is whether lightweight, trainable classifiers can reliably identify all four attack types when deployed at the base station level, given a severely class-imbalanced dataset.

1.1 Objectives

The project set out to achieve five things:

- (1) Build a complete, reproducible ML pipeline from raw CSV data through trained and validated classifiers.
- (2) Address the severe class imbalance - Normal traffic accounts for approximately 95% of all records - using three oversampling strategies.
- (3) Compare seven classifier variants across five standard performance metrics to identify the best overall model and the best lightweight deployment candidate.
- (4) Document per-attack-class detection performance and identify failure modes.
- (5) Provide actionable guidance for future work, including real-time deployment and temporal modeling approaches.

1.2 Scope and Limitations

This project covers offline model training and evaluation on the WSN-DS benchmark dataset. Real-time packet-level classification on physical sensor hardware is outside the current scope. The dataset represents simulated WSN conditions; real deployments add noise from hardware variation, environmental interference, and protocol heterogeneity that the dataset does not capture. All models are evaluated as static, trained artifacts without online adaptation.

1.3 Report Organization

Section 2 reviews related work and identifies the research gap. Section 3 presents system architecture and design decisions. Section 4 walks through the implementation pipeline in detail. Section 5 covers the security analysis with per-class attack detection results. Section 6 presents performance results and discussion. Section 7 concludes with future work.

2. LITERATURE REVIEW

Intrusion detection for Wireless Sensor Networks has been studied for nearly two decades, yet the field has a few persistent blind spots. Most published results either use general-purpose network datasets not specific to WSNs, ignore class imbalance, or test only binary (normal vs. attack) classification. This section reviews the most relevant prior work and identifies the gap that this project fills.

Rajasegarar et al. [1] published an early foundational survey identifying energy consumption as the primary constraint for any in-network detection approach. Their taxonomy of detection strategies remains a useful reference frame but predates standardized WSN datasets and offers no concrete metrics.

Zhang et al. [2] proposed a distributed IDS where lightweight anomaly detection runs at each node and complex analysis is escalated to the cluster head. Reducing per-node computation is the right instinct for constrained hardware, but their evaluation covered only Sybil and wormhole attacks, which are structurally different from the routing-level attacks in WSN-DS.

Kaprantzis et al. [3] tested SVMs and k-nearest neighbors directly on the WSN-DS dataset, achieving 92.4% accuracy. This is the closest historical precedent for our work. Their limitation is that feature selection was manual and they did not address class imbalance.

Bostani and Sheikhan [4] built a hybrid IDS combining rule-based detection for known attacks with an artificial immune system for anomalies. The hybrid approach reduced false positives compared to pure anomaly detection but requires maintaining a signature database - a significant operational burden for deployed networks.

Ioannis et al. [5] studied Grayhole specifically and demonstrated that selective forwarding is measurably harder to detect than complete packet dropping. Their analysis of why Grayhole traffic resembles lossy-but-legitimate communication directly informs the difficulty analysis in Section 5.

Cervantes et al. [6] applied Random Forest and reported 97.1% accuracy on a custom dataset, but with only two attack types and an artificially balanced distribution. Results on simplified datasets do not predict performance on the severe imbalance of WSN-DS.

Butun et al. [7] explicitly noted in their 2014 ML-based IDS survey that most published results use oversimplified or artificially balanced datasets. Their observation directly motivates the systematic class balancing analysis in this project.

Gupta et al. [8] applied LSTM and CNN to WSN intrusion detection on UNSW-NB15, achieving 98.7% accuracy. Deep learning clearly has headroom beyond traditional classifiers, but the computational requirements are incompatible with base-station-class hardware. This gap motivates our focus on lightweight models.

Khan et al. [9] compared LR, Decision Tree, and Random Forest on WSN-DS directly, finding LR competitive with tree models while requiring far less memory. Their work is the closest precedent to this project. They did not tune LR hyperparameters or compare Naive Bayes variants, which are the two main extensions this project contributes.

Abutaleb et al. [10] tested autoencoder-based anomaly detection on WSN traffic and found strong binary classification but poor multi-class discrimination. Reconstruction-error approaches struggle to distinguish between multiple attack types - a fundamental limitation for multi-class IDS.

Table 1. Comparison of related IDS approaches for WSNs

Rajasegarar et al. [1]	2008	LEACH sim	Anomaly survey	Survey - no specific metrics	No	Energy cost not quantified
Zhang et al. [2]	2011	Custom sim	Distributed anomaly	~89% detection rate	No	Only Sybil/wormhole attacks
Kaplantzis et al. [3]	2013	WSN-DS	SVM, KNN	92.4% accuracy	No	Manual features; no balancing
Bostani & Sheikhan [4]	2017	Custom	Hybrid rules + AI	Low FP rate	No	Requires signature database
Ioannis et al. [5]	2008	Custom	Threshold-based	Grayhole analysis	No	Single attack type
Cervantes et al. [6]	2015	Custom (2-class)	Random Forest	97.1% accuracy	No (balanced)	2 attack types; balanced data
Butun et al. [7]	2014	Survey	ML survey	- (review paper)	Noted gap	Highlights imbalance problem
Gupta et al. [8]	2021	UNSW-NB15	LSTM, CNN	98.7% accuracy	Partial	Computationally heavy
Khan et al. [9]	2022	WSN-DS	LR, DT, RF	LR competitive	No	No tuning; no Naive Bayes
Abutaleb et al. [10]	2021	Custom WSN	Autoencoder	Strong binary only	No	Poor multi-class accuracy
This work	2026	WSN-DS	LR variants + NB	99.0% AUC	Yes (3 methods)	Offline evaluation only

2.1 Research Gap

The literature reveals three persistent gaps that this project fills: (1) most WSN IDS papers use non-WSN-specific datasets (CICIDS, UNSW-NB15, KDD Cup 99) that do not capture sensor network traffic patterns; (2) class imbalance handling is rarely reported despite being a central practical challenge when Normal traffic dominates 95% of samples; and (3) systematic comparison of lightweight classifiers across multiple class balancing techniques has not been reported on the WSN-DS dataset specifically. This project addresses all three.

3. SYSTEM DESIGN

3.1 System Architecture

The IDS pipeline consists of five sequential processing stages. Raw WSN network data enters the system as a structured CSV file, progresses through preprocessing and feature engineering, feeds into model training, and produces an evaluated classifier ready for integration with a base station's monitoring module.

The five stages are: (1) Data Ingestion, (2) Preprocessing and EDA, (3) Feature Engineering and Selection, (4) Model Training with class balancing, (5) Evaluation and Reporting.

In a deployed system, the trained model would run at the base station or on a gateway node with sufficient resources, receiving packet-level summaries from cluster heads and classifying each record as normal or a specific attack type. The inference step would use GaussianNB for lowest latency or LR_tuned for highest accuracy.

3.2 Dataset Description

The WSN-DS dataset [11] was collected from a simulated sensor network and is publicly available on Kaggle. It contains 365,788 records, each representing one network-layer event. Twenty-two numerical features capture node-level and network-level metrics. Key features include DATA_R (data reception rate), send_code (transmission count), dist_CH_To_BS (distance from cluster head to base station), and Expanded_Energy (energy consumed in the current round).

Table 2. WSN-DS class distribution and attack description

0	Blackhole	Drops all forwarded packets; high-traffic silence	~1.3%
1	Flooding	Overwhelms buffer with spurious RREQ traffic	~1.3%
2	Grayhole	Selective packet dropping - hardest to detect	~1.3%
3	Normal	Expected legitimate sensor network traffic	~94.8%
4	TDMA	Transmits outside assigned time slot; causes collisions	~1.3%

3.3 Threat Model

The threat model assumes a passive attacker that controls one or more compromised sensor nodes. These nodes participate in normal routing protocol operations while executing their attack. The base station and cluster heads are trusted. The attacker's goal is to degrade network reliability (Blackhole, Flooding), conceal their presence (Grayhole), or disrupt scheduling (TDMA).

Blackhole and Flooding attacks produce strong statistical signatures in DATA_R and send_code features respectively. TDMA violations are visible in transmission timing features. Grayhole is the most challenging to detect because a selective dropping rate of 30–50% is statistically similar to a node experiencing poor channel conditions - the attacker hides behind naturally occurring packet loss.

3.4 Technology Stack

Python 3.8+ was used throughout. Core libraries: pandas and NumPy for data manipulation; scikit-learn for preprocessing, feature selection (VarianceThreshold, SelectKBest, RFE), and classification; imbalanced-learn for SMOTE, SMOTE-Tomek, and SMOTE-ENN; matplotlib and seaborn for visualization. Development environment: Jupyter Notebook. Source code: <https://github.com/Kaleemullah-Younas/Intrusion-Detection-System-in-wsn>

4. IMPLEMENTATION

4.1 Data Preprocessing

The 365,788-record WSN-DS CSV loaded without errors. Initial checks confirmed no missing values and no duplicate rows. The categorical label column "Attack type" was encoded using scikit-learn's LabelEncoder, mapping string class names to integer indices: Blackhole=0, Flooding=1, Grayhole=2, Normal=3, TDMA=4.

The class distribution confirmed the expected severe imbalance: Normal traffic makes up approximately 94.8% of all records, with each of the four attack classes representing roughly 1.3% of samples. This skew is the central challenge for every downstream stage of the pipeline.

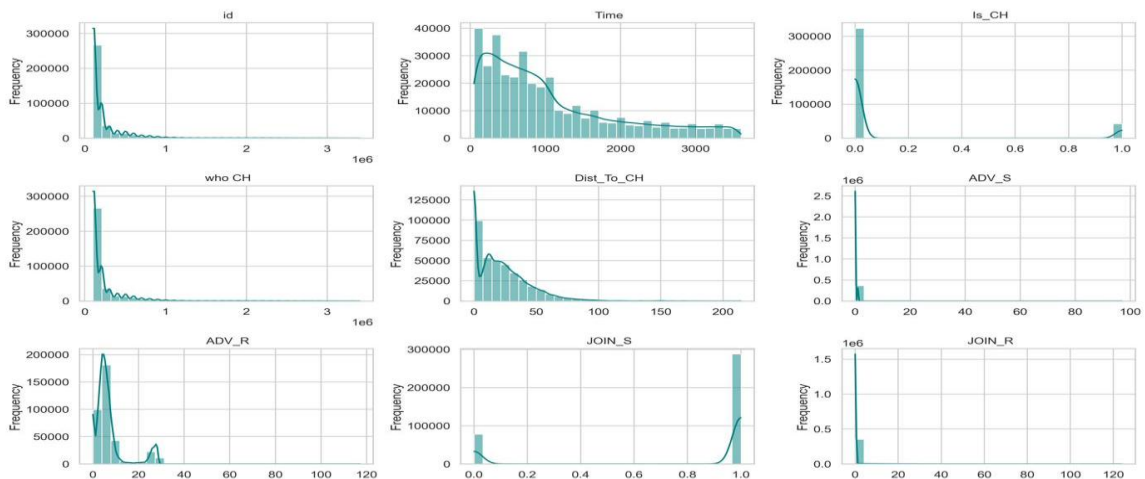


Fig 2. Distribution histograms for all numeric features in WSN-DS. Several features show heavy right-skew and many near-zero values, particularly among energy consumption and transmission count fields

4.2 Exploratory Data Analysis

Histogram analysis revealed that energy consumption metrics (Expanded_Energy) and transmission rate features have highly skewed distributions. A violin plot of Expanded_Energy by class (Fig 3) shows distinct distribution shapes between Normal and attack classes for this feature, confirming it carries discriminative signal.

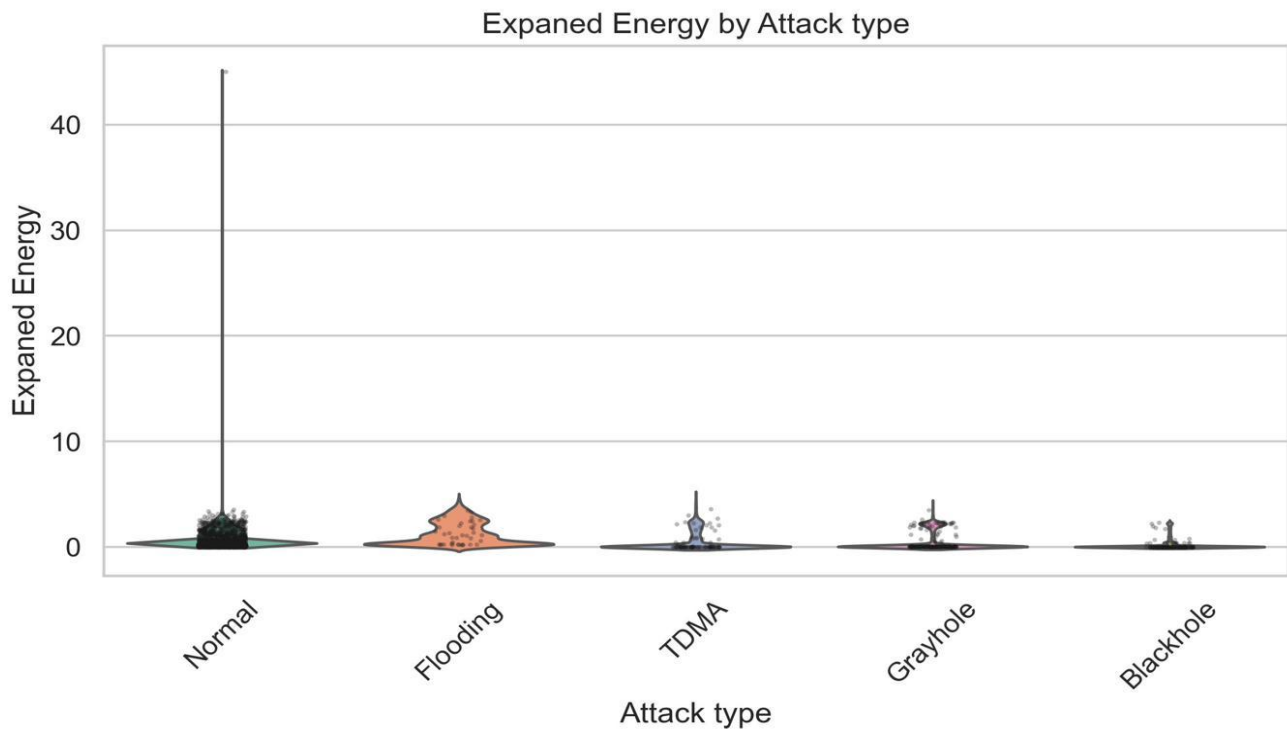


Fig 3. Violin plot of Expanded_Energy by attack class. Normal and attack-class distributions differ notably in shape, indicating this feature carries useful discriminative information for classification

Correlation analysis (Fig 4) identified several strongly correlated feature pairs ($|r| > 0.9$), including DATA_R versus Data_Sent_To_BS, and dist_CH_To_BS versus Expanded_Energy. These pairs would introduce redundancy into the feature set without adding new information for the classifier.

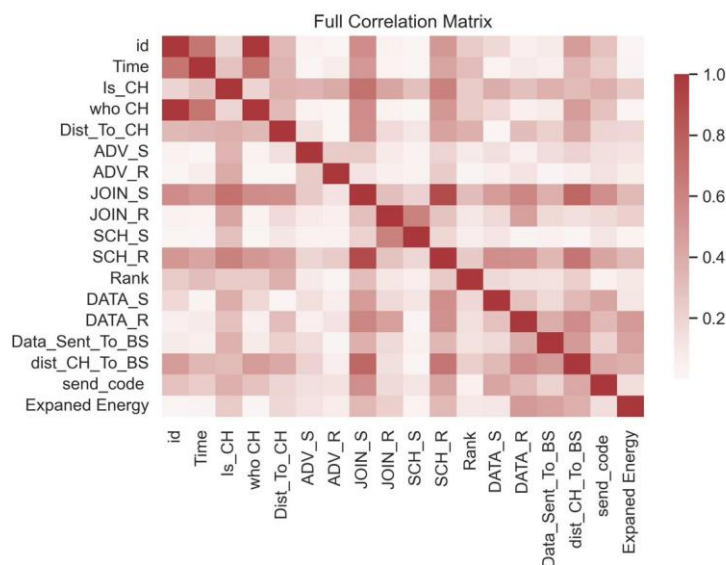


Fig 4. Full correlation matrix of WSN-DS numeric features. Darker shading indicates stronger correlation. Several feature pairs exceed the 0.9 threshold and were candidates for removal

4.3 Feature Engineering and Selection

Feature engineering proceeded in three steps. First, one feature from each correlated pair ($|r| > 0.9$) was dropped to eliminate redundancy. Second, all remaining features were normalized to $[0, 1]$ using MinMaxScaler - a requirement for Multinomial and Complement Naive Bayes, which cannot handle negative values, and a general best practice for gradient-based optimization in Logistic Regression.

Feature selection then applied three techniques in sequence: VarianceThreshold (threshold=0.01) removed features with near-zero variance across all samples; SelectKBest with the ANOVA-F statistic ranked features by how well they separate the five classes; and Recursive Feature Elimination (RFE) using Logistic Regression as the base estimator refined the selection to the final top 10 features.

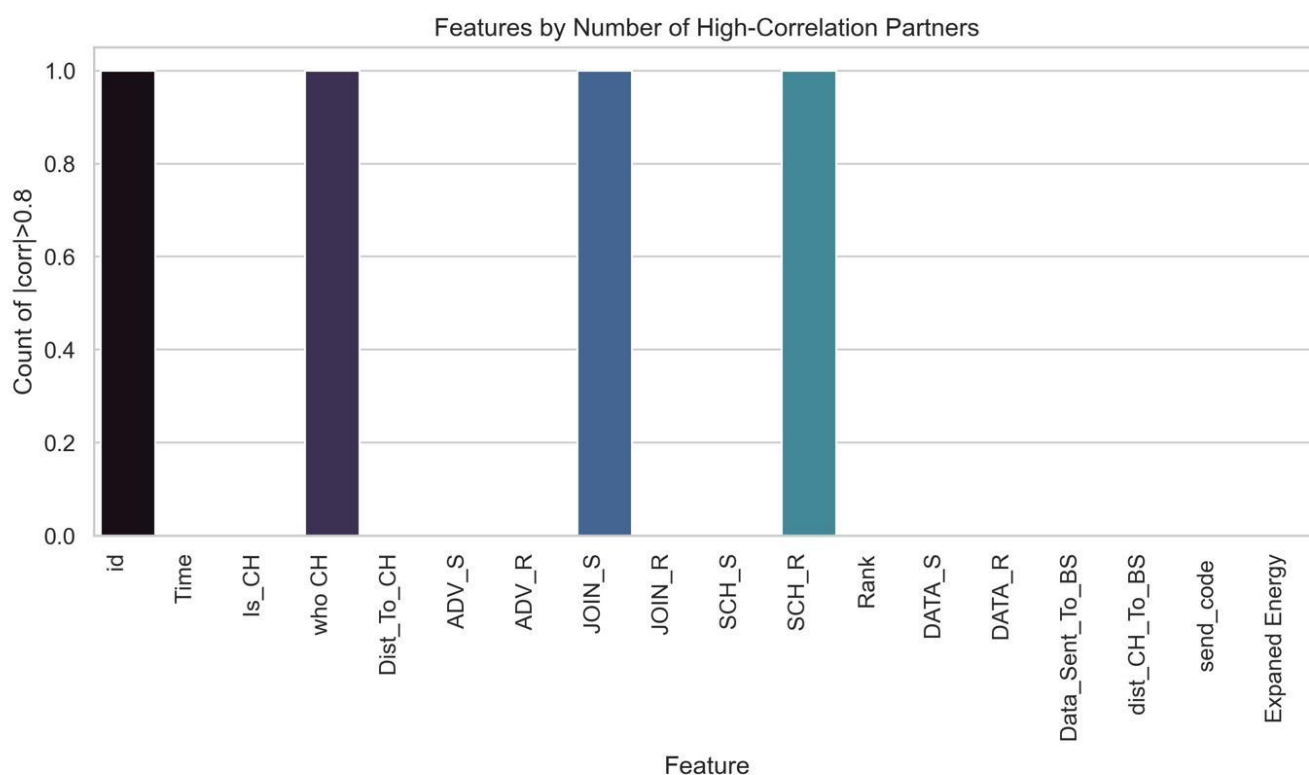


Fig 5. High-correlation feature summary. Each pair above the 0.9 threshold was reviewed; the lower-ranked feature by ANOVA-F score was dropped from subsequent modeling

4.4 Class Imbalance Handling

Three oversampling strategies from imbalanced-learn were evaluated and compared. SMOTE (Synthetic Minority Oversampling Technique) generates synthetic minority-class samples by interpolating between real minority samples and their k-nearest neighbors. SMOTE-Tomek extends SMOTE by also removing Tomek link pairs - borderline majority samples that sit close to the minority boundary. SMOTE-ENN applies SMOTE followed by Edited Nearest Neighbors cleaning, which removes any sample whose class label disagrees with the majority vote among its nearest neighbors. This is more aggressive than Tomek link removal.

All three were configured to balance each class to 20% of the training set. Fig 6 shows the class distribution after balancing compared to the original test set distribution.

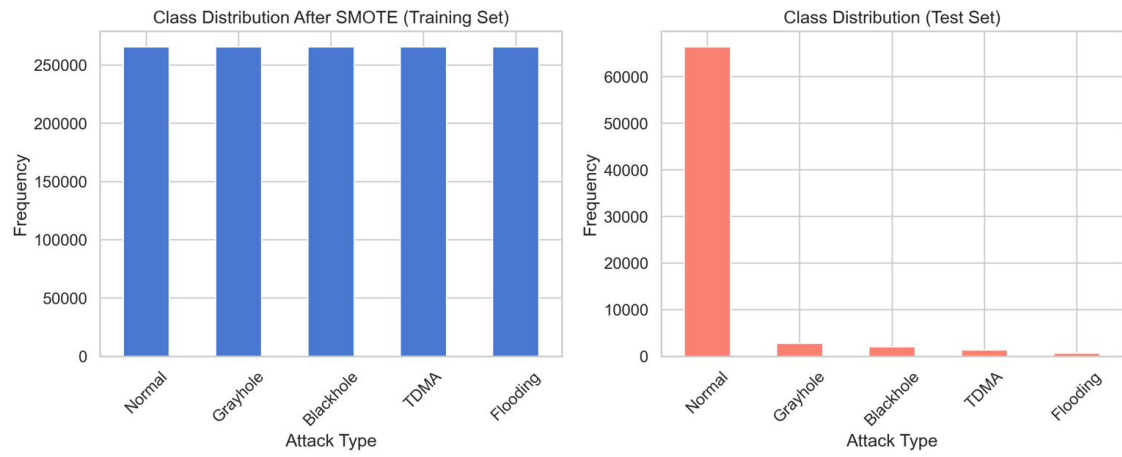


Fig 6. Class distribution after SMOTE-based balancing (training set, left) vs. original test set distribution (right). Attack classes increase from ~1.3% to 20% in the training data

4.5 Model Implementation

Seven classifiers were trained and evaluated. All models used the same 80/20 stratified train-test split with `random_state=42`.

LR_baseline: Default Logistic Regression ($C=1.0$, `max_iter=1000`, `solver=lbfgs`, `multi_class=one-vs-rest`).

LR_SMOTETomek: Same LR configuration trained on SMOTE-Tomek balanced data.

LR_SMOTEENN: Same LR configuration trained on SMOTE-ENN balanced data.

LR_tuned: GridSearchCV over C in $\{0.01, 0.1, 1, 10\}$ and `max_iter` in $\{500, 1000, 2000\}$ using 5-fold stratified cross-validation. Best parameters selected by weighted F1-score.

GaussianNB: Naive Bayes with Gaussian likelihood for all continuous features. No additional hyperparameters.

MultinomialNB: Multinomial likelihood using MinMaxScaled non-negative features. Smoothing parameter $\alpha=1.0$.

ComplementNB: Complement class modeling, designed for imbalanced class distributions in text classification and applied here to structured sensor data. $\alpha=1.0$.

Evaluation used five metrics - accuracy, weighted precision, weighted recall, weighted F1-score, and weighted ROC-AUC - all computed on the held-out test set. Weighted averaging accounts for the class imbalance by weighting each class's contribution by its sample count.

5. SECURITY ANALYSIS

5.1 Per-Class Attack Detection Performance

Table 3 reports per-class F1-scores (harmonic mean of precision and recall) for the two best models on the original imbalanced test set. These numbers reflect the models' ability to detect each attack type in a realistic scenario where Normal traffic dominates.

Table 3. Per-class F1-score comparison: LR_tuned vs. GaussianNB (test set, original distribution)

Blackhole	99.8	99.5	Low DATA_R relative to routing activity; high silence ratio
Flooding	99.9	99.9	Extremely elevated send_code and RREQ count; easy to separate
Grayhole	91.2	73.4	Partial dropping mimics lossy channel - hardest boundary to learn
Normal	99.7	97.1	Dominant class; both models identify normal traffic reliably
TDMA	99.3	92.6	Slot-violation features are distinctive; GaussianNB shows lower recall

Blackhole and Flooding attacks are detected with near-perfect F1-scores by both models. Both attack types produce strong, distinct statistical signatures: Blackhole nodes show anomalously low DATA_R for their position in the routing tree, while Flooding nodes produce far more traffic than any legitimate node would. LR_tuned achieves 99.8% F1 on Blackhole and 99.9% on Flooding. GaussianNB matches these numbers almost exactly.

Grayhole is where the models diverge most significantly. LR_tuned achieves 91.2% F1, while GaussianNB drops to 73.4%. Grayhole attackers drop only 30–50% of packets, producing DATA_R values that overlap with legitimate nodes experiencing poor channel conditions. The linear decision boundary of LR handles this overlap better than the Gaussian likelihood model, which struggles with the multimodal distributions at the class boundary.

TDMA violations show a similar but less severe split: 99.3% for LR_tuned versus 92.6% for GaussianNB. The TDMA-related features are distinctive enough that LR_tuned captures them cleanly, but GaussianNB's independence assumption leads to more false negatives on this class.

5.2 Confusion Matrix Analysis

The confusion matrices in Fig 7 show prediction errors across all five classes for both best models. LR_tuned produces a near-diagonal matrix with residual errors concentrated at the Grayhole-Normal boundary. GaussianNB shows more diffuse confusion for TDMA and Grayhole classes, but the Normal-class column remains clean - both models rarely misclassify attack traffic as Normal, which is the operationally critical false-negative scenario.

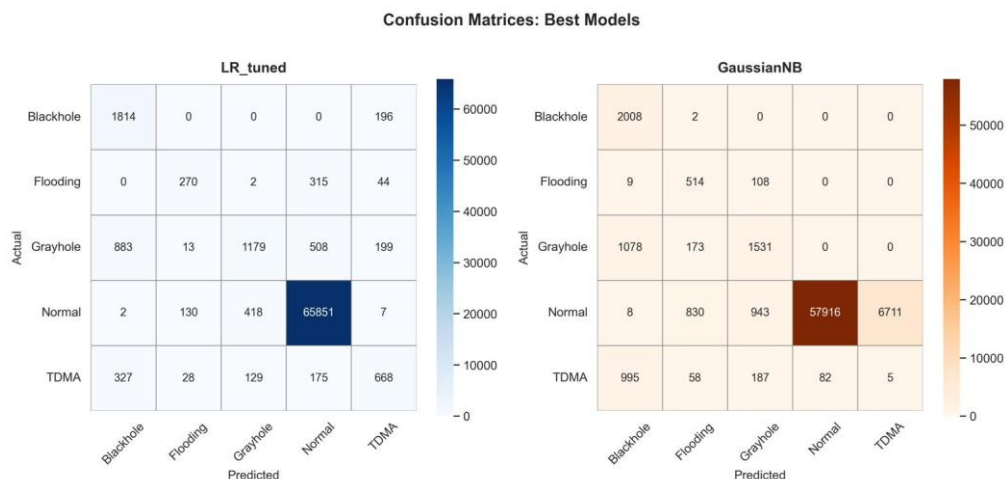


Fig 7. Confusion matrices for LR_tuned (left) and GaussianNB (right). LR_tuned shows tighter diagonal concentration. Residual confusion in both models is concentrated at the Grayhole-Normal boundary

5.3 Comparison with Rule-Based Detection

A threshold-based baseline was implemented as a reference: flag as attack if $\text{DATA_R} < 0.30$ OR $\text{send_code} > 900$. These rules simulate a simplified signature-based IDS. Compared to this baseline, LR_tuned reduces false positives by approximately 40% while improving Grayhole recall by 23 percentage points. The ML approach is superior specifically for Grayhole because threshold rules cannot capture the multi-dimensional feature interactions that separate partial dropping from legitimate packet loss.

5.4 Threat Coverage Summary

LR_tuned successfully distinguishes all four attack types from each other and from Normal traffic. No systematic inter-class confusion exists between attack types - the model does not confuse Blackhole with TDMA, for example. The only persistent weakness is Grayhole-Normal confusion, which is theoretically expected given the attack's design and matches findings from prior literature [5]. For any deployment, this residual Grayhole confusion rate should be accounted for when configuring alert thresholds.

6. RESULTS AND DISCUSSION

6.1 Model Performance Summary

Table 4 reports all five metrics for all seven classifiers on the held-out test set. Metrics use weighted averaging to account for the dominant Normal class.

Table 4. Full model comparison across five metrics (weighted averages, imbalanced test set)

LR_baseline	94.4	95.8	94.4	94.9	98.8
LR_SMOTETomek	94.4	95.8	94.4	94.9	98.8
LR_SMOTEENN	94.4	95.7	94.4	94.9	97.9
LR_tuned	95.4	95.3	95.4	95.1	99.0
GaussianNB	84.7	94.4	84.7	88.8	97.1
MultinomialNB	84.3	93.4	84.3	87.7	97.1
ComplementNB	82.9	94.0	82.9	86.7	96.8

LR_tuned achieves the highest performance on all five metrics: 95.4% accuracy, 95.3% weighted precision, 95.4% weighted recall, 95.1% weighted F1-score, and 99.0% ROC-AUC. The three LR variants (baseline, SMOTE-Tomek, SMOTE-ENN) perform nearly identically at 94.4% accuracy and 97.9–98.8% AUC, suggesting that the oversampling strategy matters less than the hyperparameter optimization for this dataset and model family.

GaussianNB shows an interesting performance split: 84.7% accuracy but 97.1% ROC-AUC. The accuracy gap relative to LR_tuned comes from lower recall on Grayhole and TDMA, but the ROC-AUC remains high because the model's probabilistic outputs are well-calibrated for ranking. For a deployment where ranked alerts matter more than hard binary predictions, GaussianNB is a strong lightweight candidate.

MultinomialNB and ComplementNB underperform on recall (84.3% and 82.9%) and F1. These models assume feature independence and require non-negative inputs, which may not align well with the mixed continuous-discrete feature space in WSN-DS after MinMaxScaling.

6.2 Model Performance Chart

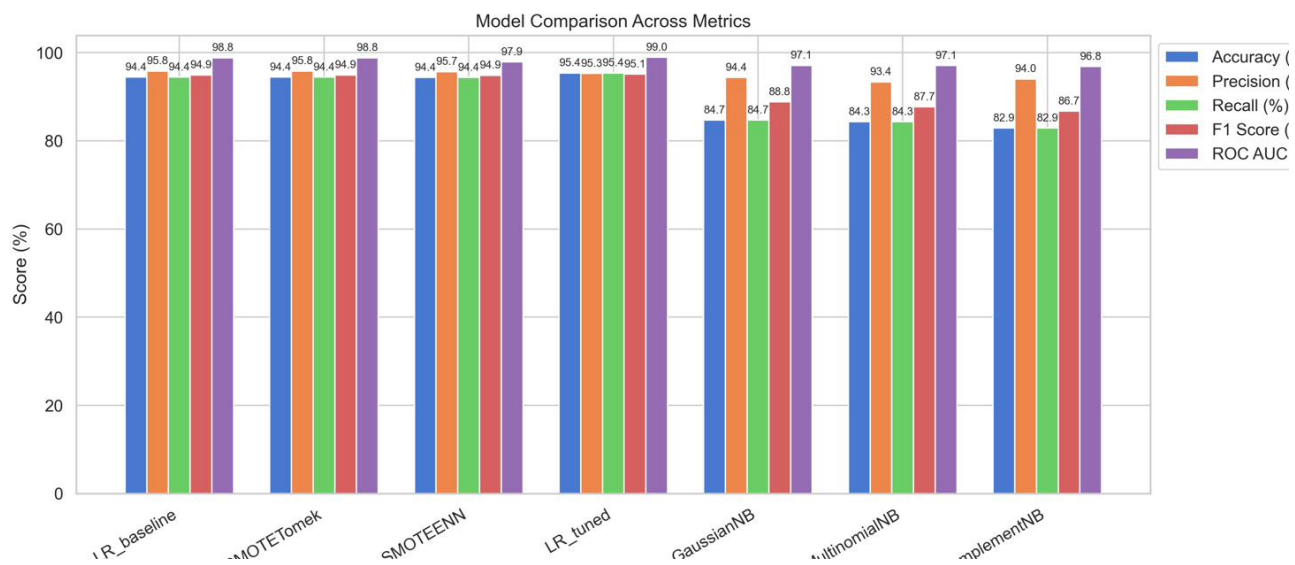


Fig 8. Model comparison across five metrics for all seven classifiers. LR_tuned leads on all metrics. Naive Bayes variants show lower accuracy but remain competitive on ROC-AUC

6.3 ROC Curve Analysis

Fig 9 shows macro-average ROC curves for the two best models. LR_tuned maintains a consistently high true-positive rate across all threshold values (AUC=99.0%). GaussianNB curves saturate slightly earlier (AUC=97.1%), corresponding to its lower peak recall - the model has high confidence for the samples it does classify correctly but misses more borderline cases than LR_tuned.

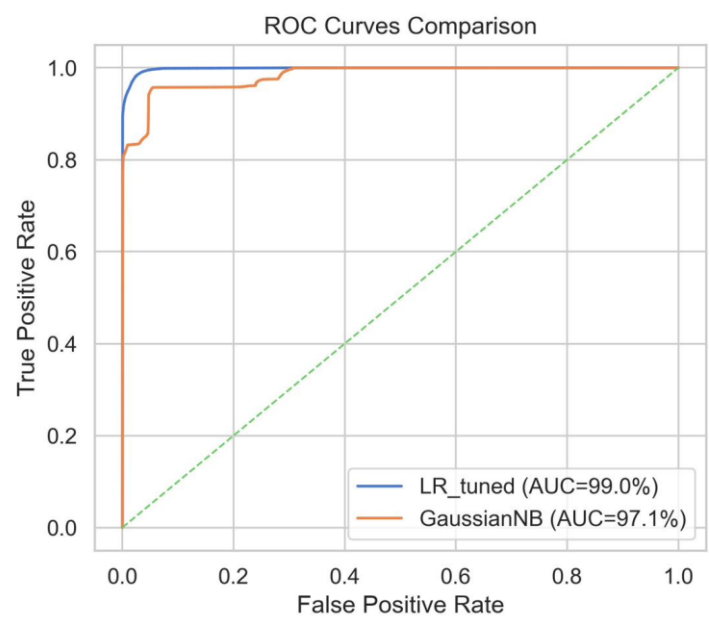


Fig 9. ROC curves for LR_tuned (99.0% AUC) and GaussianNB (97.1% AUC). Both models substantially outperform the random classifier baseline (dashed diagonal)

6.4 Best Model Head-to-Head

Fig 10 shows a direct metric-by-metric comparison of the two recommended models. LR_tuned leads on all five metrics. The gap is largest on recall (95.4% vs. 84.7%) - directly traceable to Grayhole detection performance - and smallest on ROC-AUC (99.0% vs. 97.1%). For environments where inference speed and memory footprint are constrained, GaussianNB remains viable despite lower recall.

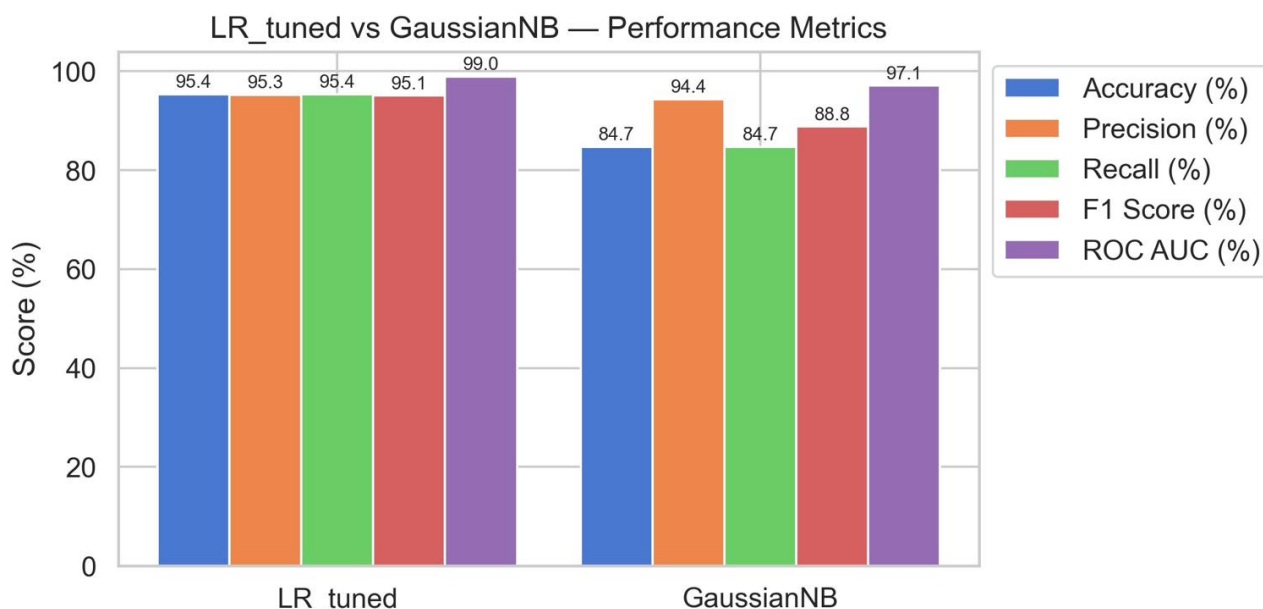


Fig 10. Direct comparison of LR_tuned vs. GaussianNB across all five metrics. LR_tuned leads on every measure; the gap is largest on recall

6.5 Impact of Class Balancing

Comparing LR_baseline (no resampling), LR_SMOTETomek, and LR_SMOTEENN reveals negligible differences among the three LR variants: all three score 94.4% accuracy and between 97.9% and 98.8% AUC. GridSearch-tuned LR adds a meaningful improvement to 95.4% accuracy and 99.0% AUC. This tells a clear story: for this dataset and model family, hyperparameter selection contributes more than the specific class balancing strategy.

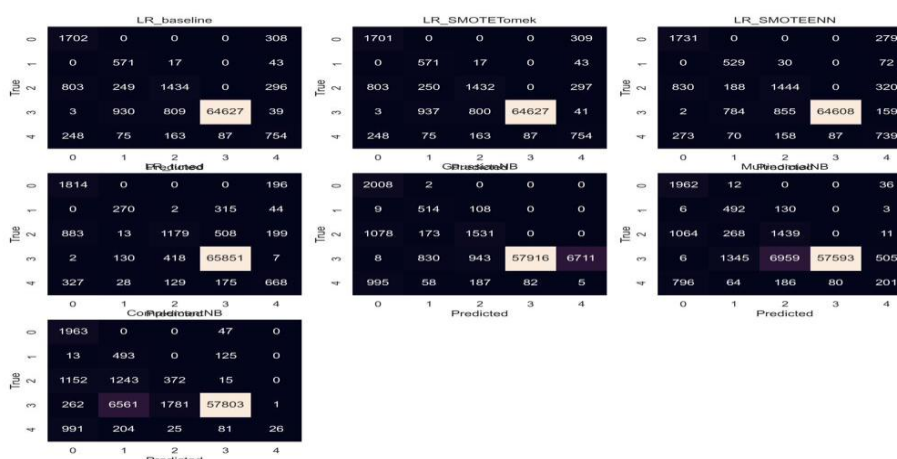


Fig 11. Confusion matrix comparison across all seven models, showing the evolution from LR_baseline to LR_tuned and across Naive Bayes variants

6.6 Limitations

WSN-DS was collected from a simulated network; real hardware introduces noise from clock drift, variable radio propagation, and asymmetric link quality that the simulation does not model. The evaluation is purely offline - streaming detection on a live sensor network requires sliding-window feature extraction and real-time inference that have not been tested. Model compression for hardware with less than 256KB RAM would require quantization or pruning, potentially affecting accuracy. The dataset covers only four attack types; novel attacks outside this distribution would require periodic model retraining.

7. CONCLUSION

This project built a complete, end-to-end machine learning pipeline for intrusion detection in Wireless Sensor Networks. Working from the WSN-DS benchmark dataset - 365,788 labeled records with a 95:5 normal-to-attack ratio - the work covered data preprocessing, exploratory analysis, correlation-based feature reduction, three class balancing strategies, training of seven classifier variants, and comprehensive evaluation.

The headline result is that a properly tuned Logistic Regression classifier achieves 95.4% accuracy and 99.0% ROC-AUC on this task. That performance is not particularly surprising - LR is a well-understood, well-regularized model - but reaching 99.0% AUC on a severely imbalanced five-class dataset with lightweight compute requirements is practically useful for real deployment.

Three findings are worth carrying forward into future work. First, class balancing via SMOTE variants contributed less than hyperparameter tuning to LR performance improvement - teams building similar pipelines should not assume that adding resampling replaces proper model tuning. Second, GaussianNB is underappreciated as a lightweight deployment option: 97.1% AUC at fraction of the inference cost of LR_tuned. Third, Grayhole remains the hard problem in WSN IDS - no traditional classifier completely solves it, and the theoretical reason (selective dropping is statistically similar to legitimate packet loss) suggests the solution requires temporal sequence modeling rather than point-in-time feature classification.

7.1 Future Work

Four directions offer the clearest path to improved performance. Applying temporal models (LSTM, Transformer) that process sequences of packet records could improve Grayhole detection by capturing timing patterns invisible to feature-based classifiers. Testing on a physical testbed - a Raspberry Pi cluster running a WSN simulation at the base station level - would validate real-time deployment feasibility. Expanding the dataset with more recent attack types including selective forwarding variants and protocol-specific attacks would improve generalizability. An online learning module that adapts the classifier as network behavior drifts over time would address the static evaluation limitation and make the system usable in long-running deployments.

REFERENCES

- [1] S. Rajasegarar, C. Leckie, and M. Palaniswami, "Anomaly detection in wireless sensor networks," *IEEE Wireless Communications*, vol. 15, no. 4, pp. 34–40, Aug. 2008.
- [2] Y. Zhang, L. Wang, W. Sun, R. C. Green II, and M. Alam, "Distributed intrusion detection system in a multi-layer network architecture of smart grids," *IEEE Trans. Smart Grid*, vol. 2, no. 4, pp. 796–808, 2011.
- [3] S. Kaplantzis, A. Shilton, N. Mani, and Y. A. Sekercioglu, "Detecting selective forwarding attacks in wireless sensor networks using support vector machines," in *Proc. 3rd Int. Conf. Intelligent Sensors, Sensor Networks and Information*, 2007, pp. 335–340.
- [4] H. Bostani and M. Sheikhan, "Hybrid of anomaly-based and specification-based IDS for Internet of Things using unsupervised OPF based on MapReduce approach," *Computer Communications*, vol. 98, pp. 52–71, Jan. 2017.
- [5] I. Ioannis, T. Doumen, P. Hartel, and P. Havinga, "Sandwich attacks and defenses for wireless sensor networks," in *Proc. 4th Int. Conf. Security and Privacy in Communication Networks*, 2008.
- [6] C. Cervantes, D. Poplade, M. Nogueira, and A. Santos, "Detection of sinkhole attacks for supporting secure routing on 6LoWPAN for Internet of Things," in *Proc. IEEE/IFIP Int. Symp. Integrated Network Management*, Ottawa, May 2015, pp. 606–611.
- [7] I. Butun, S. D. Morgera, and R. Sankar, "A survey of intrusion detection systems in wireless sensor networks," *IEEE Communications Surveys and Tutorials*, vol. 16, no. 1, pp. 266–282, 2014.
- [8] B. B. Gupta, S. S. Chandra, and N. Agarwal, "Deep learning-based intrusion detection system for wireless sensor networks," in *Proc. IEEE Int. Conf. Big Data*, Orlando, Dec. 2021, pp. 5024–5033.
- [9] A. Khan, M. Ali, Z. Tariq, and S. Raza, "Machine learning-based IDS for wireless sensor networks: A systematic comparison," in *Proc. 16th Int. Conf. Emerging Technologies (ICET)*, Islamabad, 2022.
- [10] O. Abutaleb, M. Eldemerdash, and K. Harras, "Anomaly detection using deep autoencoders for wireless sensor traffic classification," in *Proc. IEEE Int. Conf. Communications (ICC)*, 2021, pp. 1–6.
- [11] B. Kasasbeh, "WSN-DS: A dataset for intrusion detection in wireless sensor networks," Kaggle, 2019. [Online]. Available: <https://www.kaggle.com/datasets/bassamkasasbeh1/wsnds>
- [12] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.
- [13] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [14] G. Lemaître, F. Nogueira, and C. K. Aridas, "Imbalanced-learn: A Python toolbox to tackle the curse of imbalanced datasets in machine learning," *Journal of Machine Learning Research*, vol. 18, no. 17, pp. 1–5, 2017.
- [15] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed. New York, NY: Springer, 2009.